

AI 工程化训练营 模块二

尹会生

目录

阶段	主题	核心工程理念
Phase 0	工程准备：AI开发环境与项目规范	环境可复现、代码可管理
Phase 1	数据工程：NLP数据处理与特征管理	数据版本化、可追溯
Phase 2	模型开发：Transformer与高效微调	模块化、参数高效、可复现
Phase 3	模型训练与实验管理	实验可追踪、超参可优化
Phase 4	模型评估与上线决策	可测性、A/B测试、因果推断
Phase 5	模型压缩与部署	轻量化、边缘部署、服务化

01 工程准备与数据工程

所有规范只为回答一个问题

如何避免凌晨3点被叫醒修环境问题？

一、为什么新人70%的工时浪费在环境问题？

1. 同事用Python 3.8跑通的代码，我在3.10报错

代价：上线延迟3天，损失\$120,000

新成员只需三步：

```
git clone https://github.com/your-org/nlp-engineering-starter
conda env create -f environment.yml
pip install -r requirements.txt
```

2. 训练时tokenizer用BPE，推理时误用WordPiece

代价：bad case暴增40%，用户投诉率+25%

关键认知：.gitignore不是技术细节，是保住饭碗的护身符

二、工程免疫三板斧： 只做保命必需的三件事

让代码在任何机器上100%复现

输出示例：

事故现场	血泪教训
S1：同事跑不通我的代码	不写Dockerfile 代价：上线前紧急修环境，背锅裁员
S2：训练推理结果不一致	用HuggingFace默认tokenizer 代价：方言识别准确率↓40%
S3：数据集突然变化	用原始文件路径 代价：A/B测试结果无效，被质疑数据造假

【工程免疫报告】

Docker构建成功（镜像体积：1.2GB，无缓存残留） 【ok】

Tokenizer一致性验证通过（方言“老子要退票” → 相同token） 【ok】

DVC数据指纹锁定（train.jsonl SHA-256：a1b2c3...） 【ok】

未清理pip缓存 → *镜像体积+300MB*（已自动修复） 【WARN】

三、数据工程避坑指南

使用 DVC 进行数据版本管理

命令	作用
dvc init	初始化DVC
dvc add data.csv	将大文件交给DVC管理（生成 .dvc 文件）
dvc push	推送到远程存储（S3/GCS/本地）
dvc pull	从远程恢复数据
dvc diff	查看数据版本差异

数据变更可追溯（git log）
支持A/B测试不同数据版本

典型 workflows

```
# 1. 添加数据
dvc add data/raw/imdb-v2.csv

# 2. 提交DVC元文件
git add data/raw/imdb-v2.csv.dvc
git commit -m "add cleaned dataset v2"

# 3. 推送数据
dvc push

# 团队成员拉取
git pull
dvc pull # 自动下载真实数据
```

三、数据工程避坑指南

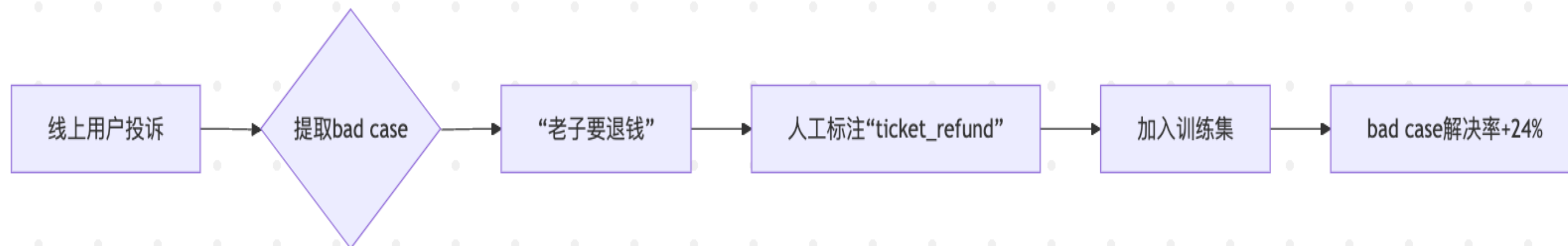
1、数据质量检查：只查致命项（拒绝过度清洗）

企业级数据检查清单（意图识别场景）

```
def check_data_quality(df):  
    assert df["text"].isna().sum() == 0, "存在空文本！" # ← 导致线上崩溃  
    assert df["intent"].nunique() > 5, "意图标签过少！" # ← 业务逻辑错误  
    assert len(df) > 1000, "样本量不足！" # ← 防止过拟合
```


三、数据工程避坑指南

2、领域语料准备：用bad case驱动



三、数据工程避坑指南

3、特征一致性保障：自动拦截灾难

在Phase 5的FastAPI服务中植入检查

```
@app.post("/predict")
```

```
async def predict(text: str):
```

```
    if not check_tokenizer_consistency(text): # ← Phase 0定义的检查
```

```
        log_critical("Tokenizer mismatch! Blocking request")
```

```
        raise HTTPException(500, "数据格式异常") # ← 拒绝脏数据污染服务
```

```
    return model.predict(text)
```

四、生存检查清单：10分钟确认工程安全

让同事/老板一眼看懂“为什么敢用我的代码”

```
# 【AI项目】工程免疫卡  Markdown
## 生存底线验证

| 验证项 | 要求 | 实测状态 | 证明方式 |
|-----|-----|-----|-----|
| **环境一致性** | Docker构建通过 | **✅ 通过** | `docker build -t app .` |
| **Tokenizer一致性** | 方言测试通过 | **✅ 通过** | `tokenizer_test.py` |
| **数据指纹锁定** | DVC状态clean | **✅ clean** | `dvc status`截图 |

## 致命风险提示
- ⚠️ ** PyTorch ** よ Phase 2
- ⚠️ ** bad case ** よ <50%
```


小结

工程规范不是束缚，而是保住饭碗的防弹衣

02 Transformer与高效微调

一、 微调相关的基础概念

微调类型	可训练参数量	资源消耗	性能	适用场景
全量微调	全部	高	最高	高性能需求、大数据集
PEFT（如LoRA）	极少（<1%）	低	高	多任务、资源受限
提示微调	极少	极低	中等	少样本、快速实验
指令微调	部分或全部	中~高	高	对话系统、任务泛化
迁移学习	视情况	中	高	跨任务/跨领域知识迁移

词元的工程指标

词元设计需综合考虑以下工程指标

问题：

- 词表大小：影响嵌入层参数量与显存占用。
- 序列长度：决定最大上下文窗口与计算复杂度。
- OOV率：反映模型对新词/专有名词的适应能力。
- 语言通用性：决定是否支持多语言混合训练。
- 可逆性与调试友好性：便于日志分析与错误溯源。

1. 为什么子词切分（BPE/WordPiece）是必须？
2. 子词切分如何影响微调效率？

一、 微调相关的基础概念

1. Tokenization：微调的起点

类型	作用	优缺点
字符级（Character-Level）	每个字符作为一个独立词元	- 缺点： - 序列长度显著增加，影响注意力机制效率。 - 语义局部性弱，需深层网络捕捉上下文。
单词级（Word-Level）	基于空格或标点划分的完整词汇	- 缺点： - 词表膨胀严重（可达数十万项），易导致内存瓶颈。 - 无法处理未见词（OOV），泛化能力差。
子词级（Subword-Level）	将词汇进一步拆分为高频子单元	- 机制： - Byte-Pair Encoding - WordPiece - Unigram Language Model - SentencePiece

2. 词元类型

3. 典型大语言模型的词元化方案对比

模型系列	词元化方法	工程特点	词表规模	多语言支持
GPT-3 / GPT-4 / ChatGPT	Byte-Level BPE	字节级编码，高鲁棒性	~50K	强
BERT / RoBERTa	WordPiece	子词切分，支持##标记	~30K	中等（主要英文）
LLaMA / LLaMA2	Byte-Level SentencePiece (BPE)	无需分词，端到端处理	~32K	强
T5	Unigram + SentencePiece	多任务统一输入格式	~32K	强

一、 微调相关的基础概念

4. 建议实践步骤:

1. 在微调前，用你的数据集抽样 100 条，测试目标模型 tokenizer 的平均 token 长度。
2. 根据最大长度选择合适的 max_length 和 batch_size。
3. 若 token 数过多，考虑：
 - 使用更粗粒度的 tokenizer（如 BERT 而非 GPT-2）
 - 对文本进行预处理（如缩写展开、术语标准化）
 - 使用序列截断或滑动窗口

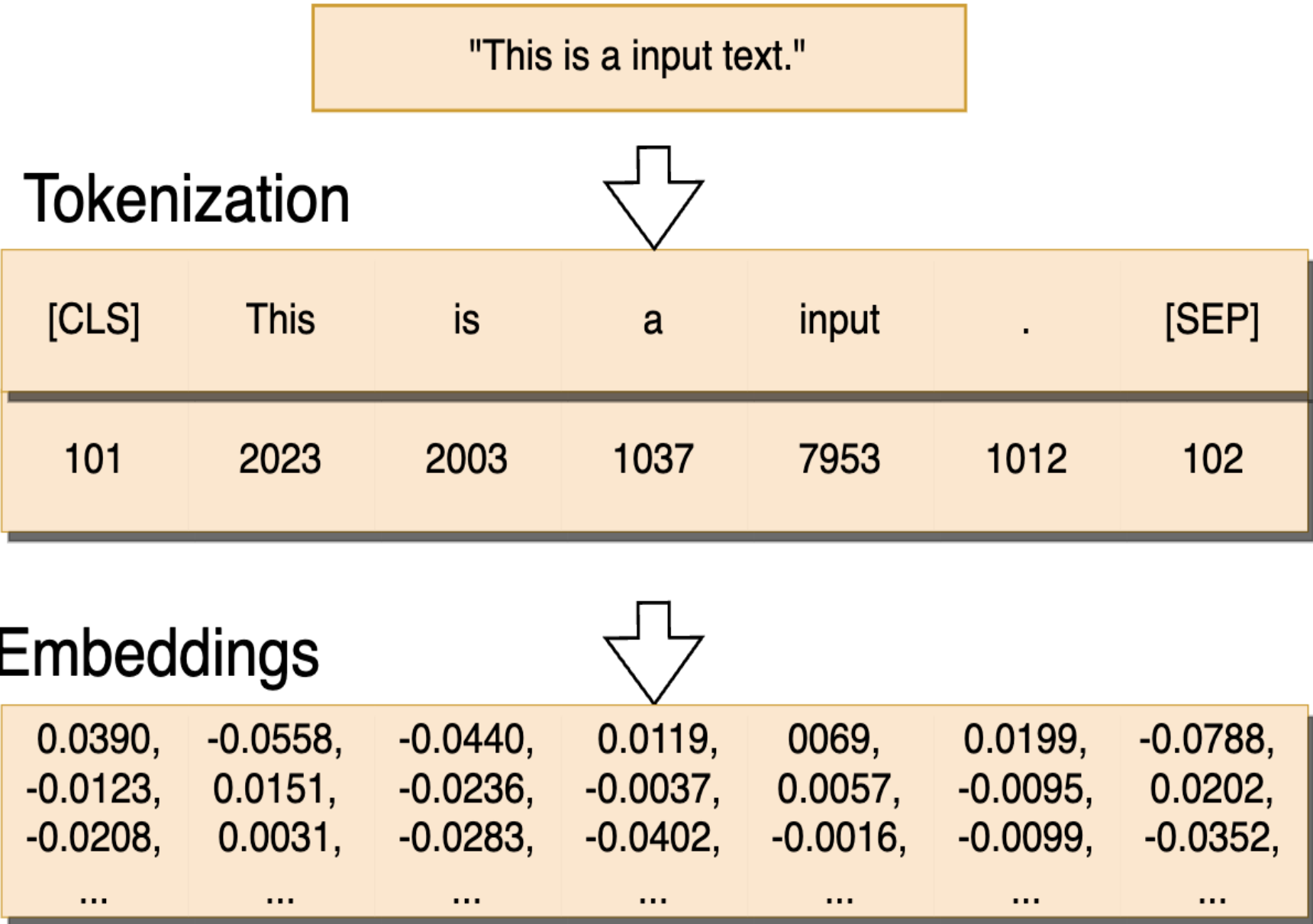
一、 微调相关的基础概念

5. Embedding与位置编码

微调流程的数据流动路径



Tokenization 是“起点”——它决定了输入有多少个 token;
而 Embedding 和位置编码是“入口”——它们决定了这些 token 如何变成模型能处理的数学表示。



一、微调相关的基础概念

6. Embedding与位置编码：为什么微调要冻结部分层？

- Embedding层：单词的“数字身份证”
- Embedding 层的参数量有多大？
 - 在 BERT-base (110M 参数) 中，embedding 层约占 35% 的总参数
- 冻结 embedding 层：保持预训练的语义空间不变，只微调上层逻辑
- Word2Vec的局限性：
 - 每个词只有一个固定向量（“bank” 在 “river bank” 和 “bank account” 中一样）
 - 无法处理多义词
 - 必须提前训练，难以融入下游任务

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- Positional Encoding: 让 Transformer “知道顺序”

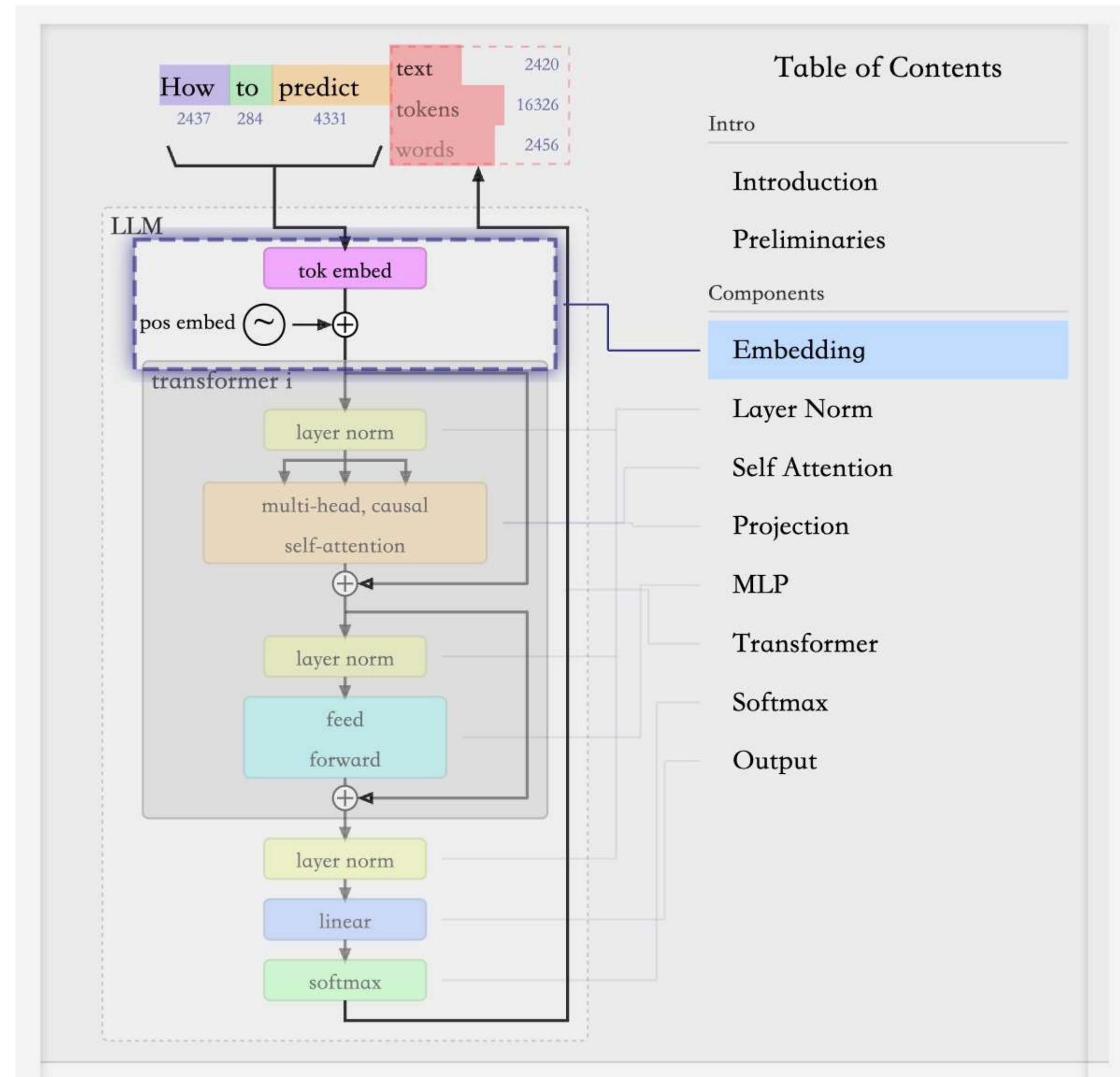
Transformer架构

Transformer由论文《Attention is All You Need》提出。

<https://arxiv.org/abs/1706.03762>

Transformer 由 Encoder 和 Decoder 两个部分组成，Encoder 和 Decoder 都包含 6 个 block。

<https://bbycroft.net/llm>



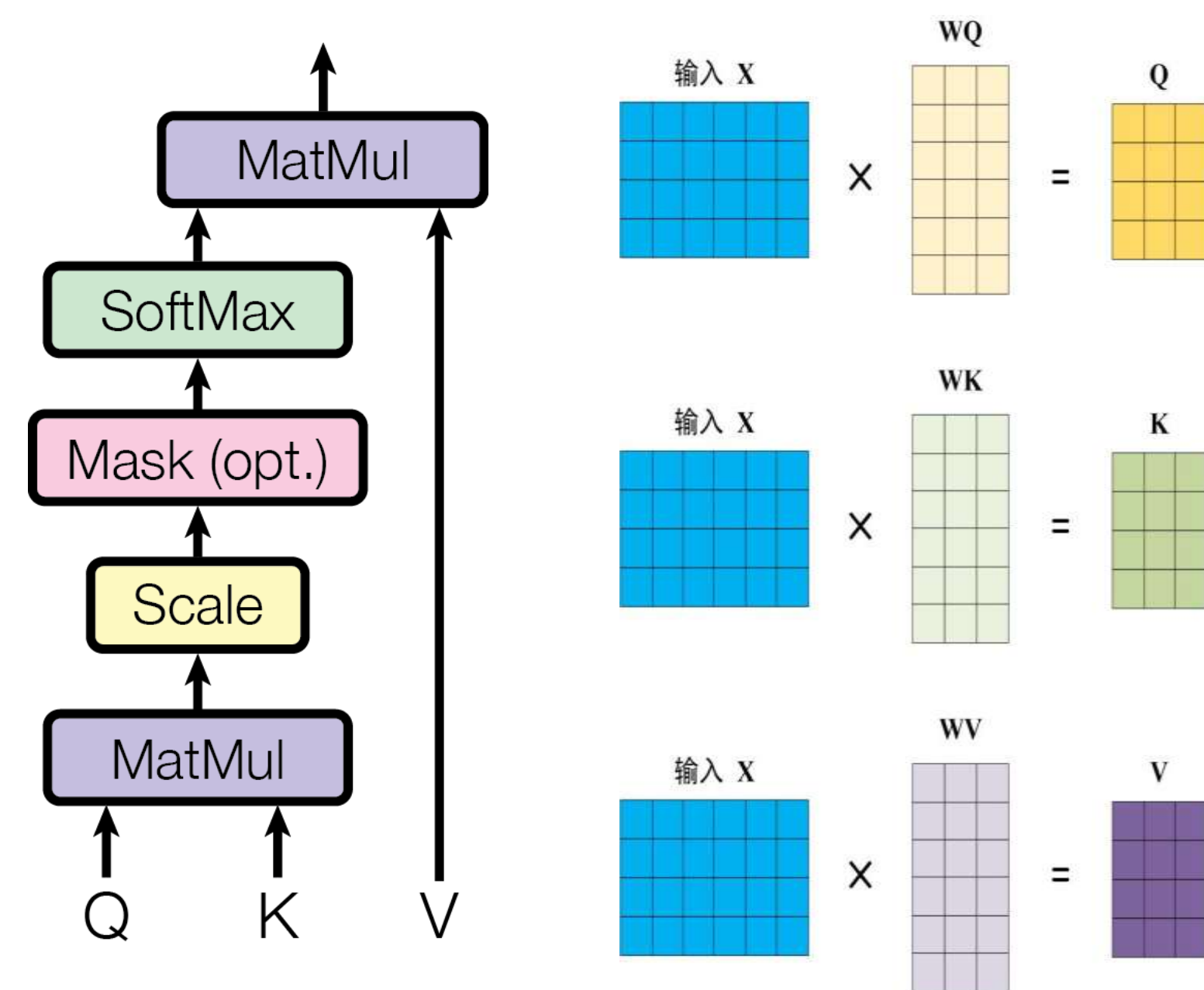
Self-Attention: 显存爆炸的根源

右图展示了 Self-Attention 的结构。该机制在计算过程中需要使用三个关键矩阵：Q（查询矩阵）、K（键矩阵）和 V（值矩阵）。在实际应用中，Self-Attention 的输入通常是词向量构成的矩阵 X（即各个单词的表示向量），或者是前一个编码器模块（Encoder block）的输出。Q、K、V 并非直接给定，而是通过对输入矩阵进行不同的线性变换（即分别乘以可学习的权重矩阵）得到的。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k 是 Q, K 矩阵的列数，即向量维度

在公式中，通过计算矩阵 Q 和 K 中各行向量之间的内积来衡量对应单词间的相关性。为了防止内积结果过大、导致后续 softmax 函数梯度消失或数值不稳定，将结果除以 K 向量维度的平方根。Q 与 K 的转置相乘后，得到一个大小为 $n \times n$ 的矩阵（其中 n 是句子中单词的数量），该矩阵的每个元素表示句子中第 i 个词与第 j 个词之间的注意力强度，反映了它们之间的关联程度。



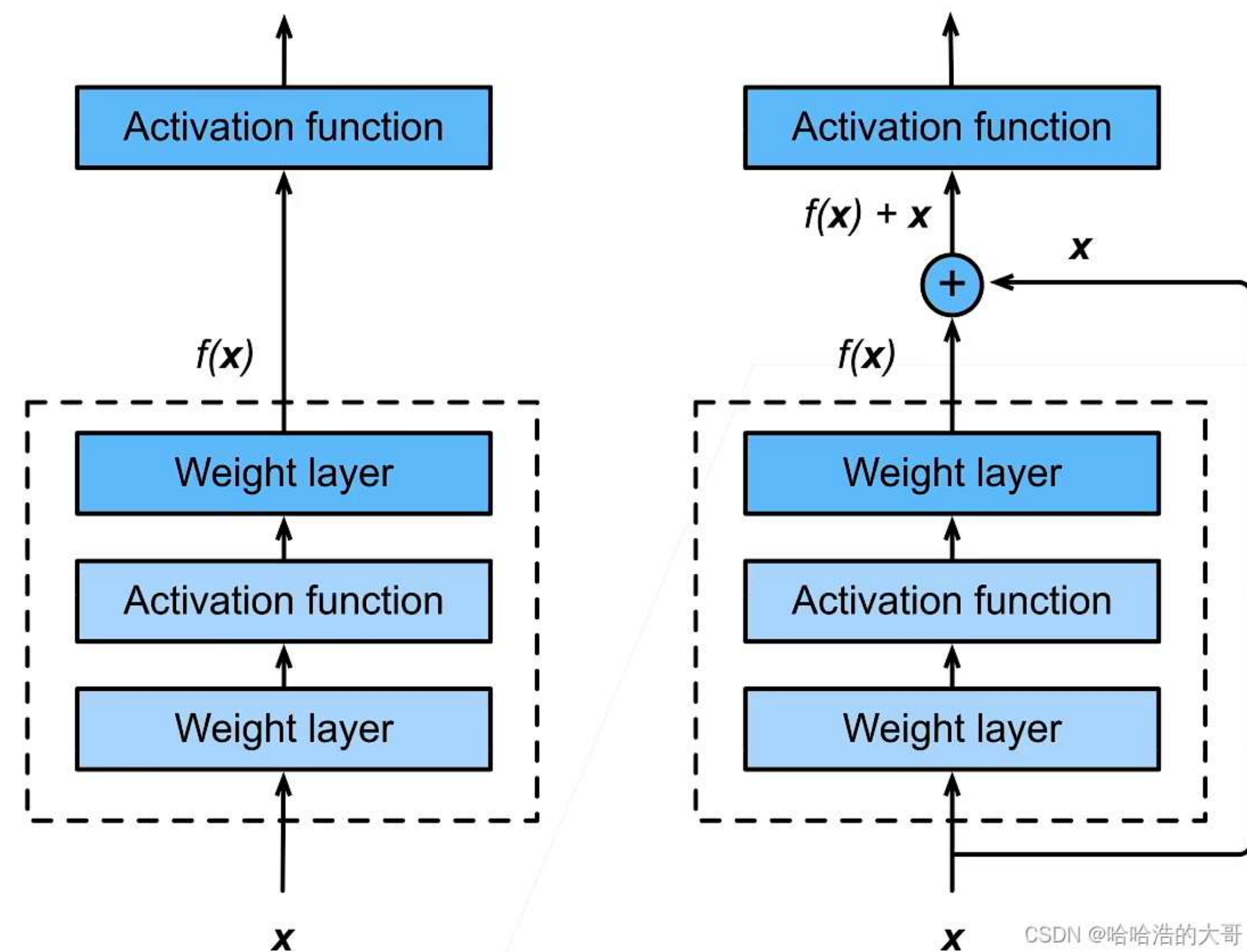
残差连接（Residual Connection） 和 层归一化（LayerNorm）

残差连接缓解梯度消失，使小数据微调成为可能

在每个 Transformer 子层（如 Self-Attention 或 FFN）之后，都有这样一条“高速公路”：

这条“+ Input”的路径，就是残差连接（Residual Connection）。

$$\frac{\partial}{\partial x}(x + F(x)) = 1 + \frac{\partial F}{\partial x}$$



CSDN @哈哈浩的大哥

Encoder-Decoder对比：为什么LoRA更适合Decoder-only模型

BERT（Encoder） vs GPT（Decoder）架构差异

维度	Decoder-only（GPT）	Encoder-only（BERT）	Encoder-Decoder（T5/BART）
是否必须用 LoRA	是（模型太大）	否（任务简单，可全调）	视情况而定
LoRA 增益是否明显	显著（节省 90%+ 显存）	一般（原模型已高效）	明显但复杂
部署灵活性	高（单头生成）	低（需额外头）	中等
结构兼容性	极佳（仅需改 Q/V）	一般（可能干扰双向性）	复杂（双侧更新）

小结

技术细节	在Lora 中如何体现
Tokenization & 序列长度 L	LoRA 不能减少 attention matrix 的L2开销，但它减少了参数更新量，间接缓解长序列微调压力
d_model（即 d）	LoRA 的核心低秩矩阵就是基于d设计的，比如 $W=W_0+\Delta W=W_0+B\cdot A$ ，其中 $B\in R^{d\times r}$, $A\in R^{r\times d}$
Self-Attention 中的 QKV 矩阵	LoRA主要就加在 Q、K、V 的投影层上，因为这些层参数多、更新代价高
显存爆炸问题	LoRA 正是为了绕过“全量微调所有参数”带来的显存和计算压力而提出的

THANKS

 极客时间 | 训练营